

MACHINE LEARNING IN FINANCE

MODULE 2 | LESSON 2

HIERARCHICAL CLUSTERING

Reading Time	70 minutes
Prior Knowledge	Clustering, unsupervised
Keywords	dendrogram, hierarchical

In the previous lesson, we explored unsupervised machine learning and saw how it differs from supervised machine learning. We also studied different clustering techniques like the k-means algorithm to find patterns in a dataset. In this lesson, we will go further using hierarchical clustering and implement it in clustering foreign exchange rates.

1. Introduction

Hierarchical clustering is an unsupervised learning model that clusters data points into hierarchies. In the last lesson, we saw how the k-means algorithm operates, and one of its limitations was that we need to know the number of clusters beforehand. In hierarchical clustering, it is not necessary to know the number of clusters before the modeling process can begin.

Hierarchical clustering can be divided into two types:

1. Agglomerative hierarchical clustering
2. Divisive hierarchical clustering

2. Agglomerative Hierarchical Clustering

Agglomerative hierarchical clustering is the most widely used (in the hierarchical family) to cluster data points based on their similarity. It uses a bottom-up approach where each data point starts as an individual cluster, and at each iteration, similar clusters are merged as we move up the hierarchy until we obtain one cluster or we identify K clusters.

2.1 Algorithm

1. Assume we have the dataset scattered in a two dimensional plane.
2. Each data point forms its own cluster at the beginning.

3. Similar clusters are merged.

4. Repeat step 2 above until a single cluster or K clusters remain(s).

Two clusters are considered similar if the distance between them is small.

2.2 Measures of Distance (Similarity)

Euclidean distance is defined as the distance between two points in a straight line. For example, if (x_1, y_1) and (x_2, y_2) are the points in a 2-dimensional space, then the Euclidean distance would be $\text{Euclidean Distance} = (x_2 - x_1)^2 + (y_2 - y_1)^2$. Euclidean Distance is applied when the variables are continuous.

Other methods of measuring distance between two points include;

- Hamming distance

We use hamming distance for text and non-numerical data, and it works by counting the number of times the coordinates in the two sets differ.

- Manhattan distance

Manhattan distance is suitable when finding the distance between different locations like finding the distance between schools. It is given by $|a-b|_1 = \sum_i |a_i - b_i|$

- Minkowski distance

Minkowski distance is given by $\left(\sum_{i=1}^n |X_i - Y_i|^p\right)^{1/p}$. When $p=1$ the Minkowski distance is equivalent to Manhattan distance, for $p=2$ the Minkowski distance is equivalent to Euclidean distance.

- Maximum distance

Maximum distance is the maximum distance we can get between two sets, that is, $|a-b|_{\infty} = \max_i |a_i - b_i|$

- Canberra distance

Canberra distance is expressed as $\sum_i \frac{|x_i - y_i|}{|x_i + y_i|}$ and terms where the denominator is zero are omitted. Canberra distance is used to compare ranked lists.

The choice of distance metric to use depends on the problem we are trying to solve.

2.3 Linkage Criterion

Once we have settled on a distance metric to apply, we need to decide where the distance will be computed. This is referred to as **linkage**.

The commonly used linkage methods are:

1. Single linkage / nearest linkage:

This is the minimum distance between a pair of data points in two different clusters given by $d(u,v) = \min(\text{dist}(u[i], v[j]))$ for data points i in cluster u and data points j in cluster v . Consider the example below

```
In [ ]: from matplotlib import pyplot as plt
        from scipy.cluster.hierarchy import dendrogram, linkage

        X = [[i] for i in [2, 8, 0, 4, 1, 9, 9, 0]]
```

From the given dataset, we obtain a single linkage using the code below.

```
In [ ]: Z = linkage(X, "single")
        fig = plt.figure(figsize=(25, 5))
        plt.suptitle(
            "Fig. 1: Dendrogram with Single Linkage",
            fontweight="bold",
            horizontalalignment="right",
        )
        dn = dendrogram(Z)
```

2. Complete Linkage:

This is the maximum distance we can get between data points in two different clusters. The mathematical representation for this is $d(u,v) = \max(\text{dist}(u[i], v[j]))$ for data points i in cluster u and data points j in cluster v . We define the complete linkage in a code as shown below:

```
In [ ]: Z = linkage(X, "complete")
        fig = plt.figure(figsize=(25, 5))
        plt.suptitle(
            "Fig. 2: Dendrogram with Complete Linkage",
            fontweight="bold",
            horizontalalignment="right",
        )
        dn = dendrogram(Z)
```

3. Average Linkage:

This is the average of the distance between each data point in one cluster to the data point in the next cluster. Its mathematical representation is as shown below $d(u,v) = \sum_{i,j} \frac{\text{dist}(u[i], v[j])}{(|u| \times |v|)}$

and just like the two linkages above we specify linkage as average in python code.

```
In [ ]: Z = linkage(X, "average")
        fig = plt.figure(figsize=(25, 5))
        plt.suptitle(
            "Fig. 3: Dendrogram with Average Linkage",
            fontweight="bold",
            horizontalalignment="right",
        )
        dn = dendrogram(Z)
```

4. Centroid Linkage:

This is the distance between the cluster centers and is given by $d(u,v) = \|c_u - c_v\|_2$ with the linkage specified as in the preceding charts.

```
In [ ]: Z = linkage(X, "centroid")
fig = plt.figure(figsize=(25, 5))
plt.suptitle(
    "Fig. 4: Dendrogram with Centroid Linkage",
    fontweight="bold",
    horizontalalignment="right",
)
dn = dendrogram(Z)
```

5. Ward's Linkage:

Ward's linkage minimizes the increase in the sum of square error at each iteration.

```
In [ ]: Z = linkage(X, "ward")
fig = plt.figure(figsize=(25, 5))
plt.suptitle(
    "Fig. 5: Dendrogram with Ward Linkage",
    fontweight="bold",
    horizontalalignment="right",
)
dn = dendrogram(Z)
```

The diagrams we have seen above are called Dendrograms and we chose the number of clusters in hierarchical clustering with the help of a dendrogram.

2.4 Dendrogram

A dendrogram is a diagram that shows the hierarchical relationship between clusters. It shows how clusters are formed at each step.

Once we have drawn the dendrogram, we slice it horizontally at the desired level then the branches that appear below the cut form individual clusters and its associated membership. Below is an example of a dendrogram with horizontal slices at different levels.

```
In [ ]: Z = linkage(X)
fig = plt.figure(figsize=(25, 5))
plt.axhline(1.2, color="red", linestyle="--")
plt.axhline(2.2, color="red", linestyle="--")
plt.axhline(0.8, color="red", linestyle="--")
plt.suptitle(
    "Fig. 6: Dendrogram with Different Horizontal Lines",
    fontweight="bold",
    horizontalalignment="right",
)
dn = dendrogram(Z)
```

3. Divisive Hierarchical Clustering

Divisive hierarchical clustering works in the opposite of agglomerative hierarchical clustering; that is, we start with a single cluster containing all the data points. Then, at each iteration, we split the data into smaller clusters until we have each data point in its own cluster.

4. Implementation of Hierarchical Clustering

In this demonstration, we use a matrix of daily returns of foreign exchange rates and perform hierarchical clustering to determine if the currencies form clusters that are related to their geographical categories.

4.1 Data Scraping

We start by installing the Yahoo finance API that will be used to download the forex data. We also import libraries for data manipulation and modeling.

```
In [ ]: # For data manipulation
# For visualization
import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns

# To fetch financial data
import yfinance as yf

plt.style.use("seaborn-darkgrid")
%matplotlib inline

from scipy.cluster.hierarchy import cophenet, dendrogram, linkage

# to perform hierarchical clustering, compute cophenetic correlation, and create
from sklearn.cluster import AgglomerativeClustering

sns.set_theme()

# Removes the limit for the number of displayed columns
pd.set_option("display.max_columns", None)

# Sets the limit for the number of displayed rows
pd.set_option("display.max_rows", 200)

# to compute distances
from scipy.spatial.distance import pdist

# to scale the data using z-score
from sklearn.preprocessing import StandardScaler
```

We now download the foreign exchange rates rendered to one U.S. dollar.

```
In [ ]: # Set the ticker as 'EURUSD=X'
forex_data = yf.download("USDEUR=X", start="2019-01-02", end="2022-06-30")
forex_data = forex_data.reset_index()
euro_df = forex_data[["Date", "Adj Close"]]
euro_df.rename(columns={"Adj Close": "euro"}, inplace=True)
```

Set the index to a datetime object

```
forex_data1 = yf.download("USDRUB=X", start="2019-01-02", end="2022-06-30")
forex_data1 = forex_data1.reset_index()
rub_df = forex_data1[["Date", "Adj Close"]]
rub_df.rename(columns={"Adj Close": "rub"}, inplace=True)

forex_data2 = yf.download("USDGBP=X", start="2019-01-02", end="2022-06-30")
forex_data2 = forex_data2.reset_index()
gbp_df = forex_data2[["Date", "Adj Close"]]
gbp_df.rename(columns={"Adj Close": "gbp"}, inplace=True)

forex_data3 = yf.download("USDJPY=X", start="2019-01-02", end="2022-06-30")
forex_data3 = forex_data3.reset_index()
jpy_df = forex_data3[["Date", "Adj Close"]]
jpy_df.rename(columns={"Adj Close": "jpy"}, inplace=True)

forex_data4 = yf.download("USDKES=X", start="2019-01-02", end="2022-06-30")
forex_data4 = forex_data4.reset_index()
kes_df = forex_data4[["Date", "Adj Close"]]
kes_df.rename(columns={"Adj Close": "kes"}, inplace=True)

forex_data5 = yf.download("USDCNY=X", start="2019-01-02", end="2022-06-30")
forex_data5 = forex_data5.reset_index()
cny_df = forex_data5[["Date", "Adj Close"]]
cny_df.rename(columns={"Adj Close": "cny"}, inplace=True)

forex_data6 = yf.download("USDKRW=X", start="2019-01-02", end="2022-06-30")
forex_data6 = forex_data6.reset_index()
krw_df = forex_data6[["Date", "Adj Close"]]
krw_df.rename(columns={"Adj Close": "krw"}, inplace=True)

forex_data7 = yf.download("USDSGD=X", start="2019-01-02", end="2022-06-30")
forex_data7 = forex_data7.reset_index()
sgd_df = forex_data7[["Date", "Adj Close"]]
sgd_df.rename(columns={"Adj Close": "sgd"}, inplace=True)

forex_data8 = yf.download("USDTHW=X", start="2019-01-02", end="2022-06-30")
forex_data8 = forex_data8.reset_index()
thw_df = forex_data8[["Date", "Adj Close"]]
thw_df.rename(columns={"Adj Close": "thw"}, inplace=True)

forex_data9 = yf.download("USDNGN=X", start="2019-01-02", end="2022-06-30")
forex_data9 = forex_data9.reset_index()
ngn_df = forex_data9[["Date", "Adj Close"]]
ngn_df.rename(columns={"Adj Close": "ngn"}, inplace=True)

forex_data10 = yf.download("USDZAR=X", start="2019-01-02", end="2022-06-30")
forex_data10 = forex_data10.reset_index()
zar_df = forex_data10[["Date", "Adj Close"]]
zar_df.rename(columns={"Adj Close": "zar"}, inplace=True)

forex_data11 = yf.download("USDMYR=X", start="2019-01-02", end="2022-06-30")
forex_data11 = forex_data11.reset_index()
myr_df = forex_data11[["Date", "Adj Close"]]
myr_df.rename(columns={"Adj Close": "myr"}, inplace=True)

forex_data12 = yf.download("USDIDR=X", start="2019-01-02", end="2022-06-30")
forex_data12 = forex_data12.reset_index()
idr_df = forex_data12[["Date", "Adj Close"]]
```

```
idr_df.rename(columns={"Adj Close": "idr"}, inplace=True)

forex_data13 = yf.download("USDTHB=X", start="2019-01-02", end="2022-06-30")
forex_data13 = forex_data13.reset_index()
thb_df = forex_data13[["Date", "Adj Close"]]
thb_df.rename(columns={"Adj Close": "thb"}, inplace=True)

forex_data14 = yf.download("USDAUD=X", start="2019-01-02", end="2022-06-30")
forex_data14 = forex_data14.reset_index()
aud_df = forex_data14[["Date", "Adj Close"]]
aud_df.rename(columns={"Adj Close": "aud"}, inplace=True)

forex_data15 = yf.download("USDNZD=X", start="2019-01-02", end="2022-06-30")
forex_data15 = forex_data15.reset_index()
nzd_df = forex_data15[["Date", "Adj Close"]]
nzd_df.rename(columns={"Adj Close": "nzd"}, inplace=True)

forex_data16 = yf.download("USDCAD=X", start="2019-01-02", end="2022-06-30")
forex_data16 = forex_data16.reset_index()
cad_df = forex_data16[["Date", "Adj Close"]]
cad_df.rename(columns={"Adj Close": "cad"}, inplace=True)

forex_data17 = yf.download("USDCHF=X", start="2019-01-02", end="2022-06-30")
forex_data17 = forex_data17.reset_index()
chf_df = forex_data17[["Date", "Adj Close"]]
chf_df.rename(columns={"Adj Close": "chf"}, inplace=True)

forex_data18 = yf.download("USDNOK=X", start="2019-01-02", end="2022-06-30")
forex_data18 = forex_data18.reset_index()
nok_df = forex_data18[["Date", "Adj Close"]]
nok_df.rename(columns={"Adj Close": "nok"}, inplace=True)

forex_data19 = yf.download("USDAUD=X", start="2019-01-02", end="2022-06-30")
forex_data19 = forex_data19.reset_index()
sek_df = forex_data19[["Date", "Adj Close"]]
sek_df.rename(columns={"Adj Close": "sek"}, inplace=True)

forex_data20 = yf.download("USDARS=X", start="2019-01-02", end="2022-06-30")
forex_data20 = forex_data20.reset_index()
ars_df = forex_data20[["Date", "Adj Close"]]
ars_df.rename(columns={"Adj Close": "ars"}, inplace=True)

forex_data21 = yf.download("USDPLN=X", start="2019-01-02", end="2022-06-30")
forex_data21 = forex_data21.reset_index()
pln_df = forex_data21[["Date", "Adj Close"]]
pln_df.rename(columns={"Adj Close": "pln"}, inplace=True)

forex_data22 = yf.download("USDPHP=X", start="2019-01-02", end="2022-06-30")
forex_data22 = forex_data22.reset_index()
php_df = forex_data22[["Date", "Adj Close"]]
php_df.rename(columns={"Adj Close": "php"}, inplace=True)

forex_data23 = yf.download("USDRON=X", start="2019-01-02", end="2022-06-30")
forex_data23 = forex_data23.reset_index()
ron_df = forex_data23[["Date", "Adj Close"]]
ron_df.rename(columns={"Adj Close": "ron"}, inplace=True)

forex_data24 = yf.download("USDHUF=X", start="2019-01-02", end="2022-06-30")
forex_data24 = forex_data24.reset_index()
huf_df = forex_data24[["Date", "Adj Close"]]
```

```

huf_df.rename(columns={"Adj Close": "huf"}, inplace=True)

forex_data25 = yf.download("USDBRL=X", start="2019-01-02", end="2022-06-30")
forex_data25 = forex_data25.reset_index()
brl_df = forex_data25[["Date", "Adj Close"]]
brl_df.rename(columns={"Adj Close": "brl"}, inplace=True)

forex_data26 = yf.download("USDCLP=X", start="2019-01-02", end="2022-06-30")
forex_data26 = forex_data26.reset_index()
clp_df = forex_data26[["Date", "Adj Close"]]
clp_df.rename(columns={"Adj Close": "clp"}, inplace=True)

forex_data27 = yf.download("USDMXN=X", start="2019-01-02", end="2022-06-30")
forex_data27 = forex_data27.reset_index()
mxn_df = forex_data27[["Date", "Adj Close"]]
mxn_df.rename(columns={"Adj Close": "mxn"}, inplace=True)

forex_data28 = yf.download("USDCOP=X", start="2019-01-02", end="2022-06-30")
forex_data28 = forex_data28.reset_index()
cop_df = forex_data28[["Date", "Adj Close"]]
cop_df.rename(columns={"Adj Close": "cop"}, inplace=True)

forex_data29 = yf.download("USDILS=X", start="2019-01-02", end="2022-06-30")
forex_data29 = forex_data29.reset_index()
ils_df = forex_data29[["Date", "Adj Close"]]
ils_df.rename(columns={"Adj Close": "ils"}, inplace=True)

forex_data30 = yf.download("USDTRY=X", start="2019-01-02", end="2022-06-30")
forex_data30 = forex_data30.reset_index()
try_df = forex_data30[["Date", "Adj Close"]]
try_df.rename(columns={"Adj Close": "try"}, inplace=True)

forex_data31 = yf.download("USDINR=X", start="2019-01-02", end="2022-06-30")
forex_data31 = forex_data31.reset_index()
inr_df = forex_data31[["Date", "Adj Close"]]
inr_df.rename(columns={"Adj Close": "inr"}, inplace=True)

```

The currencies are combined into one dataframe that will form the basis of our modeling.

```

In [ ]: from functools import reduce

df_currencies = reduce(
    lambda x, y: pd.merge(x, y, on="Date", how="outer"),
    [
        kes_df,
        ars_df,
        php_df,
        myr_df,
        ils_df,
        cop_df,
        euro_df,
        ngn_df,
        huf_df,
        ron_df,
        cny_df,
        rub_df,
        clp_df,
        sgd_df,
        twd_df,

```

```

        krw_df,
        idr_df,
        thb_df,
        inr_df,
        pln_df,
        try_df,
        brl_df,
        mxn_df,
        zar_df,
        gbp_df,
        jpy_df,
        aud_df,
        nzd_df,
        cad_df,
        chf_df,
        nok_df,
        sek_df,
    ],
)
df_currencies.set_index("Date", inplace=True)

df_currencies.head(10)

```

We have 911 rows of daily rates and 32 currencies in our dataframe.

```
In [ ]: df_currencies.shape
```

```
In [ ]: df_currencies.info()
```

```
In [ ]: df_currencies.describe()
```

The next step is to find missing values in our dataset.

```
In [ ]: df_currencies.isna().sum()
```

We impute the missing value by propagating the observations from the last day to the missing day data.

```
In [ ]: df_currencies.fillna(method="ffill", inplace=True)
df_currencies.isna().sum()
```

Since the currencies differ a lot in value range, we scale them to ensure they have the same mean and unit variance.

```
In [ ]: # Scaling the data to bring it to the same scale
```

```

sc = StandardScaler()
subset_scaled_df = pd.DataFrame(
    sc.fit_transform(df_currencies),
    columns=df_currencies.columns,
)
subset_scaled_df.head()

```

```
In [ ]: # Generate simple percentage returns to calculate the respective betas
```

```
df_returns = (df_currencies / df_currencies.shift(-1)) - 1
df_returns.head()
```

4.2 Model Training

```
In [ ]: subset_scaled_df.drop(subset_scaled_df.tail(1).index, inplace=True)
```

```
In [ ]: subset_scaled_df.tail()
```

We now find the **cophenetic correlation** using a combination of distance metrics and linkage methods. A cophenetic correlation measures the dendrogram's ability to preserve the initial unmodeled data points. The higher the cophenetic correlation, the better the performance of a dendrogram in preserving the original distance.

The cophenetic correlation is defined as:
$$c = \frac{\sum_{i < j} (Y_{ij} - y)(Z_{ij} - z)}{\sum_{i < j} (Y_{ij} - y) \sum_{i < j} (Z_{ij} - z)}$$

where

- Y_{ij} is the distance between i and j in Y .
- Z_{ij} is the cophenetic distance between i and j .
- y and z are the mean of Y and Z respectively.

```
In [ ]: # list of distance metrics
distance_metrics = ["euclidean", "chebyshev", "cityblock"]

# list of linkage methods
linkage_methods = ["single", "complete", "average", "weighted"]

high_cophenet_corr = 0
high_dm_lm = [0, 0]

for dm in distance_metrics:
    for lm in linkage_methods:
        Z = linkage(subset_scaled_df.T, metric=dm, method=lm)
        c, coph_dists = cophenet(Z, pdist(subset_scaled_df.T))
        print(
            "Cophenetic correlation for {} distance and {} linkage is {}".format(
                dm.capitalize(), lm, c
            )
        )
        if high_cophenet_corr < c:
            high_cophenet_corr = c
            high_dm_lm[0] = dm
            high_dm_lm[1] = lm
```

We now identify the distance metric and linkage method that give the highest cophenetic correlation.

```
In [ ]: # printing the combination of distance metric and linkage method with the highest
print(
    "Highest cophenetic correlation is {}, which is obtained with {} distance an
    high_cophenet_corr, high_dm_lm[0].capitalize(), high_dm_lm[1]
```

```
)
)
```

Since the Euclidean distance gives us the best performance, we combine it with different linkage methods and evaluate which combination will improve our performance.

```
In [ ]: # List of linkage methods
linkage_methods = ["single", "complete", "average", "centroid", "ward", "weighted"]

high_cophenet_corr = 0
high_dm_lm = [0, 0]

for lm in linkage_methods:
    Z = linkage(subset_scaled_df.T, metric="euclidean", method=lm)
    c, coph_dists = cophenet(Z, pdist(subset_scaled_df.T))
    print("Cophenetic correlation for {} linkage is {}".format(lm, c))
    if high_cophenet_corr < c:
        high_cophenet_corr = c
        high_dm_lm[0] = "euclidean"
        high_dm_lm[1] = lm
```

Again, we print the linkage method that gives the highest cophenetic correlation.

```
In [ ]: # printing the combination of distance metric and linkage method with the highest
print(
    "Highest cophenetic correlation is {}, which is obtained with {} linkage".format(
        high_cophenet_corr, high_dm_lm[1]
    )
)
```

Let us plot the graphs of different dendrograms to visualize how clusters vary with different linkage methods.

```
In [ ]: # List of linkage methods
linkage_methods = ["single", "complete", "average", "centroid", "ward", "weighted"]

# Lists to save results of cophenetic correlation calculation
compare_cols = ["Linkage", "Cophenetic Coefficient"]

# to create a subplot image
fig, axs = plt.subplots(len(linkage_methods), 1, figsize=(15, 30))
# `plt.suptitle`()

# We will enumerate through the list of linkage methods above
# For each linkage method, we will plot the dendrogram and calculate the cophenetic correlation
for i, method in enumerate(linkage_methods):
    Z = linkage(subset_scaled_df.T, metric="euclidean", method=method)

    dendrogram(Z, ax=axs[i])
    axs[i].set_title(f"Dendrogram ({method.capitalize()} Linkage)")
    fig.suptitle(
        "Fig. 11: Dendrograms Plots", fontweight="bold", horizontalalignment="right"
    )

    coph_corr, coph_dist = cophenet(Z, pdist(subset_scaled_df.T))
    axs[i].annotate(
        f"Cophenetic\nCorrelation\n{coph_corr:0.2f}",

```

```
(0.80, 0.80),
xycoords="axes fraction",
)
```

To find the components of each cluster, we can train the data as shown below, then find the components as demonstrated in the steps below.

```
In [ ]: HCmodel = AgglomerativeClustering(n_clusters=3, affinity="euclidean", linkage="a
HCmodel.fit(subset_scaled_df.T)
```

```
In [ ]: df_currencies_t = df_currencies.T
subset_scaled_df_t = subset_scaled_df.T

subset_scaled_df_t["HC_Clusters"] = HCmodel.labels_
df_currencies_t["HC_Clusters"] = HCmodel.labels_
```

```
In [ ]: cluster_profile = df_currencies_t.groupby("HC_Clusters").mean()
```

```
In [ ]: cluster_profile.head()
```

```
In [ ]: cluster_profile
```

We finally see the composition of 3 different clusters and the outlier which will be the column 'HC Cluster' that was engineered above.

```
In [ ]: # Let's see the names of the securities in each cluster
for cl in df_currencies_t["HC_Clusters"].unique():
    print(
        "The",
        df_currencies_t[df_currencies_t["HC_Clusters"] == cl].iloc[:, 0].nunique
        "Securities in cluster",
        cl,
        "are:",
    )
    print(df_currencies_t[df_currencies_t["HC_Clusters"] == cl].iloc[:, 0].index)
    print("-" * 100, "\n")
```

```
In [ ]: df_currencies_t[df_currencies_t["HC_Clusters"] == 0].iloc[:, 0].index
```

Cluster one has the Kenya shilling, Argentine peso, Colombian peso, Nigerian naira, Russian ruble, Chilean peso, Indonesian rupiah, Indian rupee, Turkish lira, Mexican peso, and South African rand.

Cluster two has the Israeli shekel, Chinese yuan, Singaporean dollar, Taiwanese dollar, British pound, Australian dollar, New Zealand dollar, Canadian dollar, Swiss franc, Norwegian krone, and Swedish krona.

Cluster three has the Philippine peso, Malaysian ringgit, Euro, Hungarian forint, Romanian leu, South Korean won, Thai bhat, Poland zloty and Japanese yen.

We can see from the clusters that most developing countries are in the first cluster while developed nations are mostly in the second cluster.

We can also draw the dendrogram using the methodology below.

```

In [ ]: hier_average = linkage(subset_scaled_df.T, method="average", metric="euclidean")

In [ ]: hier_ward = linkage(subset_scaled_df.T, method="ward", metric="euclidean")

In [ ]: hier_comp = linkage(subset_scaled_df.T, method="complete", metric="euclidean")

In [ ]: # Change the chart style...
plt.style.use("fivethirtyeight")

In [ ]: df_currencies_t = df_currencies.T

plt.figure(figsize=(10, 10))
plt.title(
    "Dendrogram of FX Clusters, Jan 2019 through Jun 2022 (Complete)", fontsize=
)
plt.xlabel("Distance", fontsize=10)
plt.ylabel("Currency", fontsize=10)
plt.suptitle(
    "Fig. 12: Dendrogram with Complete Linkage",
    fontweight="bold",
    horizontalalignment="right",
)
dendrogram(
    hier_comp,
    orientation="right",
    # leaf_rotation=90.,
    leaf_font_size=20,
    labels=df_currencies_t.index.values,
    color_threshold=3,
)
plt.yticks(fontsize=6)
plt.show()

```

5. Conclusion

In this lesson, we have studied and implemented hierarchical clustering. In the next lesson, we will focus on another unsupervised learning method that is used to reduce the number of variables in a dataset.

References

1. Murtagh, Fionn, and Pedro Contreras. "Algorithms for Hierarchical Clustering: An Overview." *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*, vol. 2, no. 1, 2012.
2. Murtagh, Fionn, and Pedro Contreras. "Algorithms for Hierarchical Clustering: An Overview, II." *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*, vol. 7, no. 6, 2017.
3. Murtagh, Fionn, and Pedro Contreras. "Methods of Hierarchical Clustering." *arXiv preprint*, arXiv:1105.0121, 2011.
4. Nielsen, Frank. "Hierarchical Clustering." *Introduction to HPC with MPI for Data Science*. Springer, 2016, pp. 195-211.

